

Estimación de ELO a partir de PGN mediante ajuste fino de DeBERTa-v3-small

1st Jesús Alejandro Salazar González

Posgrado en Ciencia e Ingeniería de la Computación, UNAM

Sede IIMAS Mérida

Yucatán, México

alejandro.salazar9812@comunidad.unam.mx

Abstract—En este trabajo se trata de estimar el puntaje Elo de jugadores de ajedrez a partir de la notación PGN de sus partidas. Se comparan dos enfoques: un modelo secuencial con word embeddings y una arquitectura preentrenada tipo Large Language Model (LLM), en este caso DeBERTa-v3-small. Se utiliza una base de datos pública que provino de lichess.org. Los resultados muestran que el modelo con word embeddings presenta sobreajuste y no generaliza adecuadamente, mientras que DeBERTa ofrece predicciones más estables aunque con errores significativos en zonas de baja densidad de datos. El mejor resultado alcanza un MAE promedio cercano a 210 puntos Elo, mostrando que es posible predecir el nivel de juego de un jugador utilizando texto pero todavía existe un margen de mejora.

I. INTRODUCCIÓN

El ajedrez es un juego de mesa que consta de un tablero de 8×8 en el cual cada jugador dispone de 16 piezas, las cuales poseen diferentes valores de importancia, se desplazan de distintas maneras dependiendo de ciertas reglas, y tienen la capacidad de capturar las piezas del oponente contrario [1]. El objetivo de este juego es dejar en jaque mate al rey rival, es decir amenazarlo con alguna de nuestras piezas y no dejarle escapatoria por medio de movimientos legales. Este juego a pesar de parecer limitado por el número de piezas y la cantidad de casillas disponibles, es en realidad muy complejo debido a la cantidad de combinaciones distintas que se pueden dar en el tablero, llegando a estimar que la complejidad del árbol del ajedrez está alrededor de 10^{120} jugadas posibles [2]. Es importante destacar que a pesar de haber una inmensa cantidad de jugadas posibles, muchas veces surgen algunos patrones que los jugadores más experimentados son capaces de reconocer.

Determinar el nivel de cada jugador es una tarea que se lleva a cabo en torneos. Por ejemplo el sistema de puntuación Elo mide la fuerza relativa de un jugador en comparación a otros. Este sistema fue adoptado oficialmente por la Federación Estadounidense de Ajedrez en 1960 y por la FIDE en 1970 [3]. Existen una gran variedad de organizaciones y portales de ajedrez que utilizan también este método para evaluar a los jugadores. Cada organización tiene su propia implementación, que puede ser diferente del sistema Elo original.

Para estudiar el ajedrez se requiere poder grabar las partidas mediante un lenguaje escrito, uno de estos es el PGN (Portable Game Notation por sus siglas en inglés) [4]. El texto de las jugadas describe los movimientos realizados en el juego. Esto

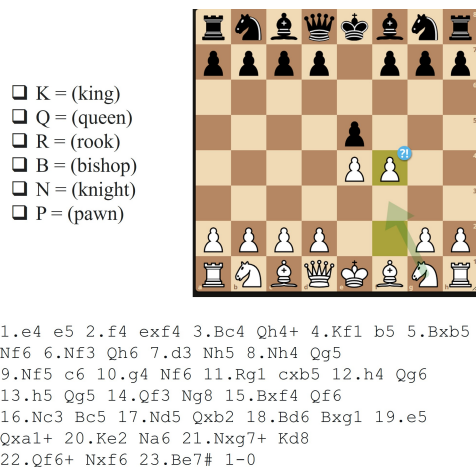


Fig. 1: Ejemplo de PGN y tablero de ajedrez

debe incluir indicadores del número de la jugada (un número seguido de cero o más puntos) y notación estándar algebraica (NEA).

Debido a la que pueden existir ciertos patrones en partidas de ajedrez, por ejemplo, los jugadores novatos tienden a mover peones en los extremos del tablero sin un plan en particular, mientras que otros con un poco más de pericia suelen atacar el centro en la apertura, surge la idea de que sea posible entrenar un modelo de procesamiento natural que sea capaz de, a partir de la notación PGN, determinar el nivel de ambos jugadores, es decir el elo del jugador de piezas blancas y negras.

II. METODOLOGÍA

En este proyecto se pretende atacar este problema mediante dos enfoques, uno mediante el uso de word embeddings y el otro mediante el uso de LLM.

En ambas técnicas se empleará una base de datos disponible en [5]. Esta conjunto de datos consta de 843230 partidas con PGN e información correspondiente al nivel de ambos jugadores, proveniente de la plataforma online lichess. Se extrajo el ELO correspondiente de cada jugador y se realizó una estandarización con $\mu = 0$ y $\sigma = 1$.

Para el caso de los word embeddings primero se realiza un preprocesamiento que consta en quedarnos únicamente con información de las piezas que se juegan, es decir nuestro

modelo no recibira la información que pueda indicar cual es el número de jugada que se encuentra el juego, sino un vector con las posiciones jugadas. En este caso se utilizara una red neuronal usando unidades GRU. La arquitectura de este modelo se compone primero de una capa de entrada capa recibe secuencias de 60 tokens enteros, los cuales representan las jugadas del PGN ya codificadas mediante un vocabulario. Luego una capa de transforma cada token entero en un vector denso de dimensión 128. Seguidamente una capa contiene una celda GRU con 128 unidades en cada dirección (hacia adelante y hacia atrás). Despues se emplea otra capa GRU bidireccional, con 64 unidades por dirección. Despues un par de capas densas con función de activación ReLU para finalmente retornar unicamente 2 valores con información del ELO del jugador de piezas blancas y negras.

En lo que compete el uso de LLM se empleo el modelo DeBERTa, el cual mejora la eficiencia del preentrenamiento de BERT y RoBERTa mediante dos ideas clave: atención desenredada y un decodificador de máscaras mejorado. A diferencia de BERT, que mezcla todo, DeBERTa separa el contenido de una palabra de su posición y los procesa de forma independiente. Esto le permite comprender mejor qué se dice y en qué parte de la oración ocurre. En este caso se utilizara el modelo deberta-v3-small [6] el cual cuenta con 44 millones de parametros para realizar un ajuste fino sobre nuestros datos de Elo.

III. RESULTADOS Y DISCUSIÓN

A. DeBERTa

Entre las primeras pruebas realizadas primero se opto por emplear unicamente 60000 partidas de ajedrez en las cuales 10 % de estas se emplearon para validación. Al realizar un ajuste fino sobre este conjunto de datos se obtuvieron las gráficas que se muestran en la figura 10. Tanto en la figura 9a y 9c se muestran el valor del Elo verdadero vs el Elo predicho por nuestro modelo, en el caso ideal se esperaria que en todos los datos convergan sobre la linea diagonal punteada marcada en rojo, pero en este caso el modelo tiende a realizar mejores predicciones para aquellos jugadores que se encuentran en un nivel intermedio, alrededor de un puntaje entre 1200 y 2000. Para visualizar mejor esto se realizan los histogramas de la figura 9b y 9d, aquí notamos que el conjunto de predicciones arroja en su mayoría predicciones que se encuentran en el intervalo mencionado anteriormente, e incluso si comparamos la desviación media entre predicciones y datos reales, esta varia alrededor de 100 puntos entre ambas. Esto nos hace sospechar que posiblemente exista un tipo de desbalanceo de clases, es decir el modelo recibió, en su mayoría, datos provenientes de ese intervalo, por lo que unicamente aprendio a predecir eso.

Para tratar de corregir el problema anterior se opta por realizar un muestreo un poco más estratificado. Para eso primeramente aplicaremos el algoritmo Kmeans sobre nuestro conjunto de datos empleando un total de 256 clases para dividir nuestro espacio, resultando en una estructura que se muestra en la figura 10a, donde se observa que existen regiones

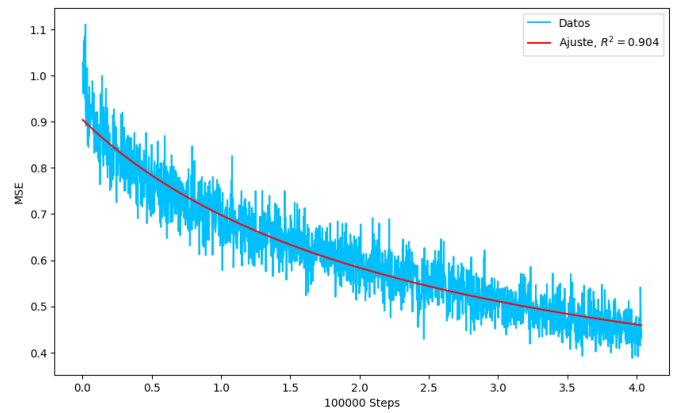


Fig. 2: Curva de aprendizaje con 3 épocas

con mayor densidad de datos y otras con mayor dispersión. Seguidamente se realiza la gráfica 10b donde se muestra la cantidad de datos por cluster, hallando que la clase con menor número de individuos es de 197. A partir de esto se realizó un muestreo por clase de 197 individuos aleatorios para crear un nuevo conjunto de datos con mayor balance, resultando en un total de 50432 muestras. Se dividió este conjunto de datos en 80 % para entrenamiento y el resto para pruebas.

Con este nuevo dataset se realizó nuevamente el fine tuning, en esta ocasión con un total de 10 épocas, obteniendo la curva de aprendizaje que se muestra en la figura 2 se muestra la curva de aprendizaje de nuestro entrenamiento. Aquí se noto que nuestro modelo tiene un tipo de comportamiento que se asemeja a una curva racional, por lo que se realiza una regresión para tratar de estimar si valdría la pena continuar el entrenamiento, ya que en esta ocasión los resultados de MAE para blancas y negras fue de 209.78 y 214.98 respectivamente. A partir de esos resultados se estimo que el entrenar con un total de 70 épocas obtendríamos una mejor convergencia, obteniendo un valor de MSE cercano a 0.18. Este ajuste fino final duro aproximadamente 25 horas.

Al finalizar el entrenamiento se observo que el MSE obtuvo valores más bajos que la estimación hecha anteriormente, por lo que se decidió emplear estos resultados (ver figura 3). En la figura 11 se muestran las distribuciones de los datos. En primera instancia puede notarse que estos nuevos ajustes lineales se acercan un poco más a los hechos en la figura 10, debido a que su pendiente es más alta, también notamos que las distribuciones de las predicciones se distribuyen un poco más a los extremos, obteniéndose una varianza más alta, por lo que en esta ocasión el modelo tiende a arrojar una mayor variedad de datos y no se centra unicamente en los jugadores con un puntaje promedio.

A pesar de lo anterior al obtener las métricas de MAE 214.46 para blancas y 220.26 para negras, los cuales son peores a los mostrados con un entrenamiento de 10 épocas. Esto apunta a que a pesar de obtener una pérdida mayor en nuestra curva de aprendizaje con 70 épocas, el modelo lo que aprendio es a disminuir su error en los datos de entrenamiento,

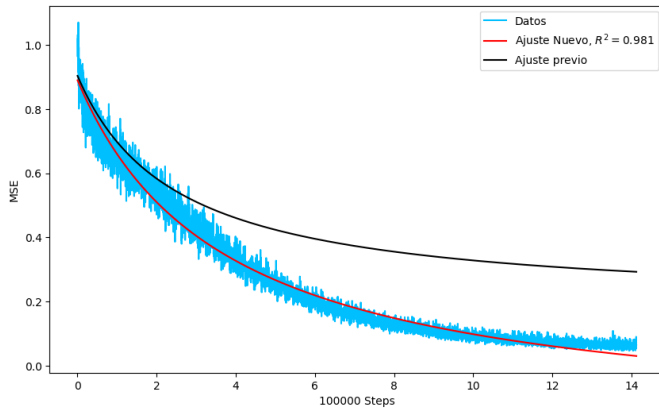


Fig. 3: Curva de aprendizaje con 10 épocas

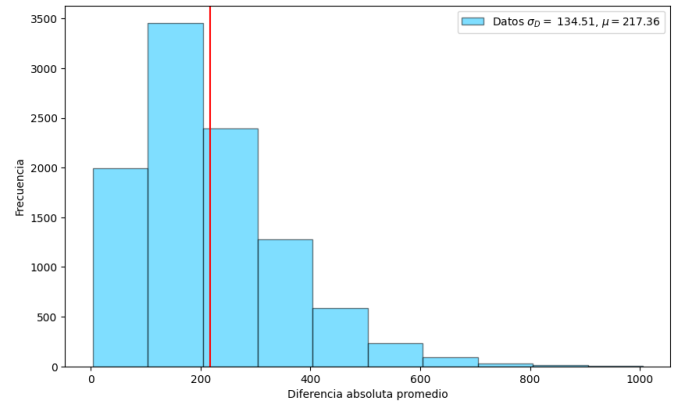


Fig. 5: Histograma MAE de resultados finales de fine tuning con DeBERTa

por lo que en esta ocasión se entre en un caso de sobreajuste.

En la figura 4 se muestra un mapa de calor donde se refleja como va variando el MAE promedio de blancas y negras por cada predicción, en los ejes coordenados se grafican los Elos verdaderos de los jugadores. De aquí puede notarse que el modelo tiende equivocarse con más severidad en valores alejados al centro de masa de la distribución de datos, ya que pues en general los clusters creados en los valores extremos tienen un región mayor, provocando así que la densidad en esa localidad sea menor y por lo tanto en esas áreas el modelo se halla entrenado con menos datos.

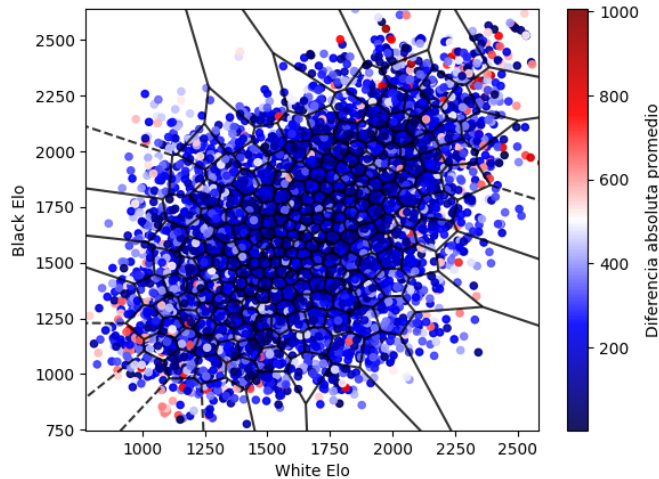


Fig. 4: Mapa de color

También es interesante destacar el amplio rango del MAE promedio, en este caso va desde 3.42 hasta 1005.85 puntos, lo cual puede sonar alarmante pero la media de estos resultados se ubican en 217.37 con una desviación estandar de 134.51, lo cual claramente es una distribución sesgada a la izquierda, y se nota que son pocos datos en los que se equivoca de manera grave (ver figura 5).

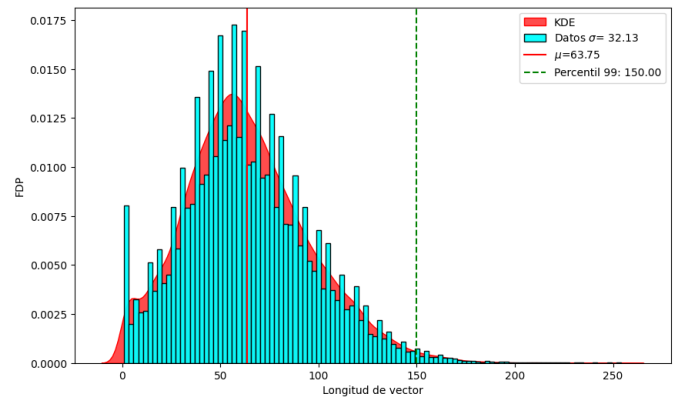


Fig. 6: Distribución longitud de partidas

B. Word Embeddings

Debido a que el PGN suele tener comprimida la información de las jugadas en la partida de ajedrez no fue necesario hacer un procesamiento tan extenso, únicamente fue necesario quitar el número de jugada en el que se encuentra actualmente los jugadores y algunos espacios extras que quedaban. Se procedió a analizar el vocabulario del conjunto de prueba y se observó que se consta con un vocabulario de 5289 palabras, las más comunes se encuentran en la figura 8.

Asimismo se muestra que las longitudes de las partidas abarcan tienen una media de 60 palabras, es decir 30 jugadas por cada jugador (figura 6). Por esta razón se considero que la longitud de los vectores al momento de tokenizar tuvieran una dimensión 60 debido a que la mayoría de las partidas abarcan este rango, y se considera que aquellas que superan este número han demostrado su capacidad de juego durante la apertura.

En la figura 7 se muestra la curva de aprendizaje para el modelo de word embeddings. En esta ocasión se resalta que al poder visualizar el comportamiento del conjunto de validación es fácil notar que se cae en un sobre ajuste, ya que la pérdida del conjunto de entrenamiento tiende a bajar de manera continua mientras que para la validación empieza

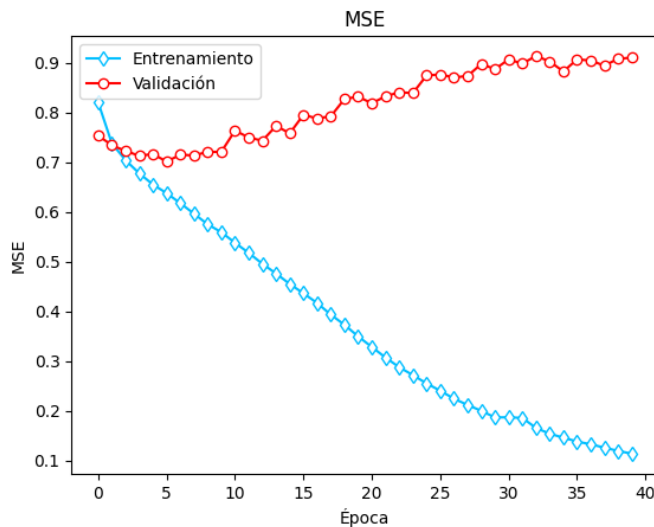


Fig. 7: Curva de aprendizaje para Word embeddings

a empeorar pasadas las 7 épocas, lo cuál nos dice que este modelo no es capaz de generalizar hacia datos no vistos.

IV. CONCLUSIONES

En este trabajo se evaluaron dos enfoques para estimar el Elo de jugadores de ajedrez a partir del texto PGN. Los resultados muestran que los modelos con word embeddings tienen dificultades para generalizar, ya que tienden a sobreajustarse después de pocas épocas. Esto sugiere que, debido a la complejidad y numerosas jugadas y combinaciones que pueden darse en el ajedrez, los métodos tradicionales no son capaces de capturar adecuadamente la naturaleza del problema.

Por otro lado, DeBERTa mostró un mejor comportamiento general, especialmente cuando se aplicó un muestreo estratificado mediante K-means para reducir el sesgo hacia partidas de nivel intermedio. Sin embargo, incluso con este ajuste, el modelo presenta errores mayores en regiones de baja densidad de datos. Además, se observó que extender demasiado el entrenamiento provoca sobreajuste, ya que el modelo reduce su pérdida en entrenamiento pero empeora en validación. Se sugiere aplicar algún tipo de mallado que tienda a dar una mayor uniformidad al muestreo de datos.

REFERENCES

- [1] "ajedrez — definición — diccionario de la lengua española — rae - asale." [Online]. Available: <https://dle.rae.es/ajedrez>
- [2] S. J. Russell, P. Norvig, M.-W. Chang, J. Devlin, A. Dragan, D. Forsyth, I. Goodfellow, J. Malik, V. Mansinghka, J. Pearl, and M. J. Wooldridge, *Artificial intelligence : a modern approach*. Pearson, 2022.
- [3] "Sistema de puntuación elo - términos de ajedrez - chess.com." [Online]. Available: <https://www.chess.com/es/terms/sistema-puntuacion-elo-ajedrez>
- [4] "Notación de juego portátil (pgn) de ajedrez - chess.com." [Online]. Available: <https://www.chess.com/terms/chess-pgn>
- [5] "Online chess games database." [Online]. Available: <https://www.kaggle.com/datasets/ironicninja/online-chess-games?resource=download>
- [6] "Deberta/readme.md at master · microsoft/deberta · github." [Online]. Available: <https://github.com/microsoft/DeBERTa/blob/master/README.md>

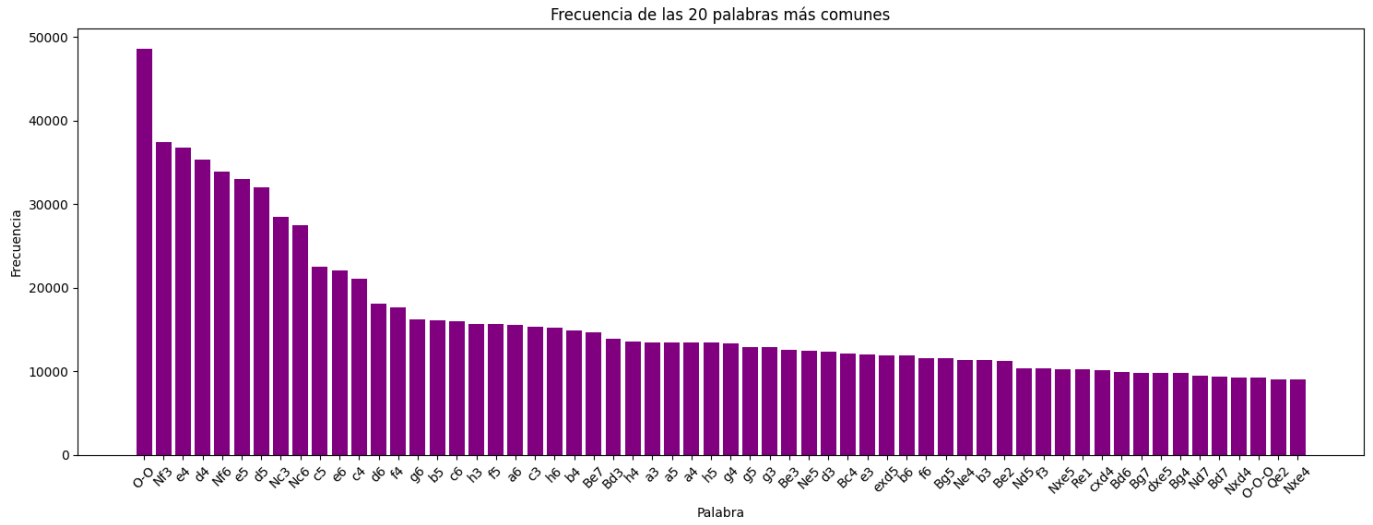
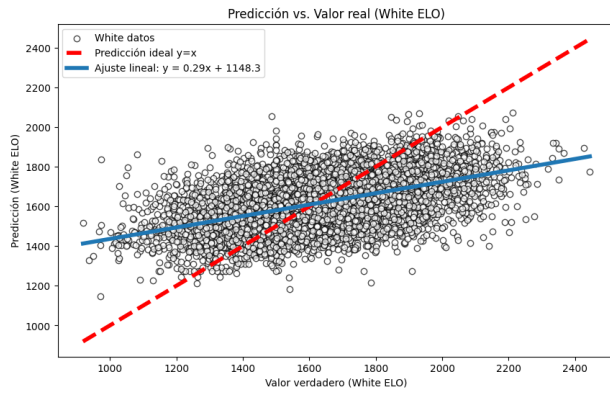
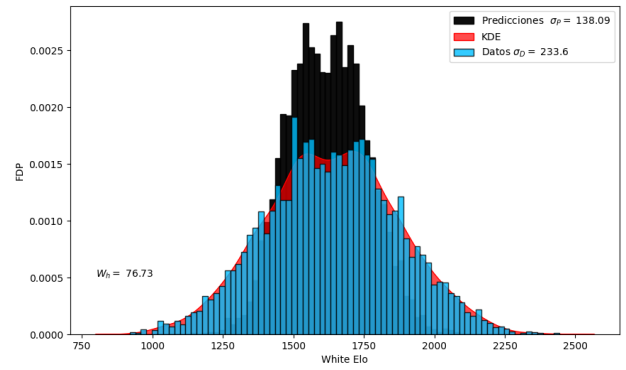


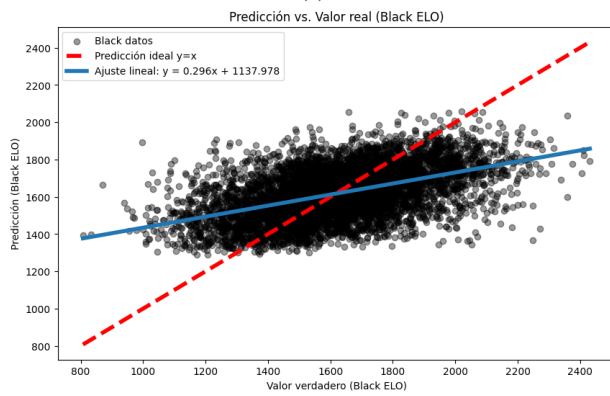
Fig. 8: Jugadas más comunes



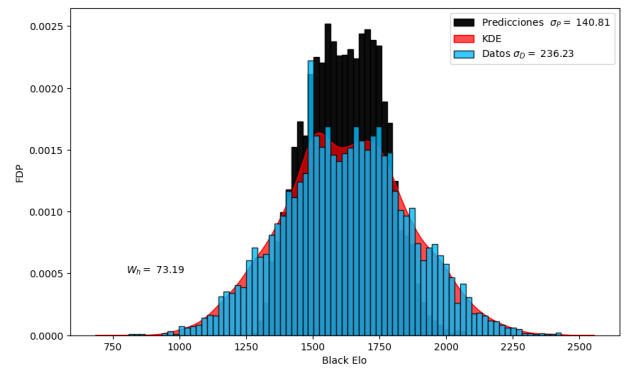
(a)



(b)

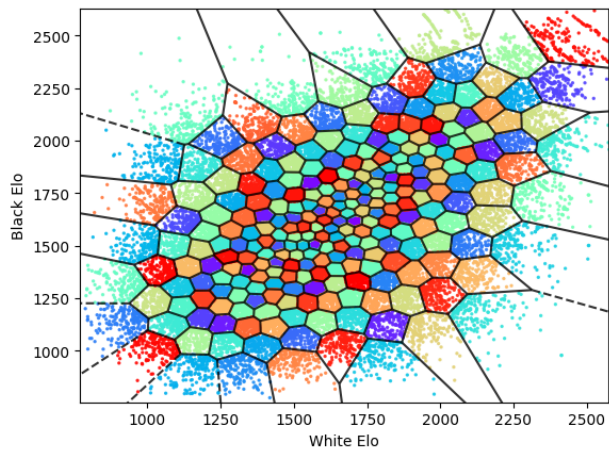


(c)

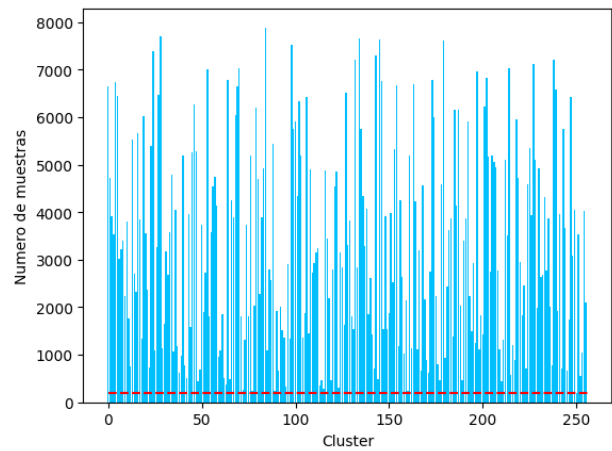


(d)

Fig. 9: Resultados obtenidos en pruebas preliminares.

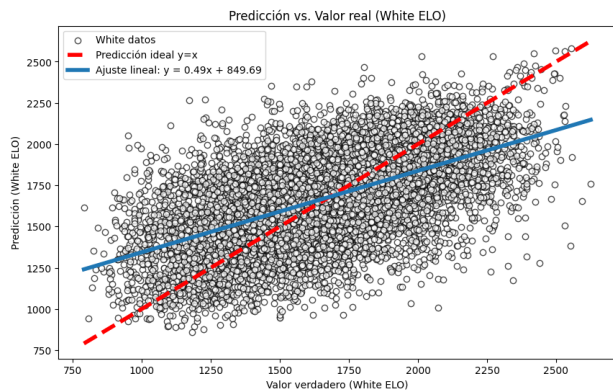


(a)

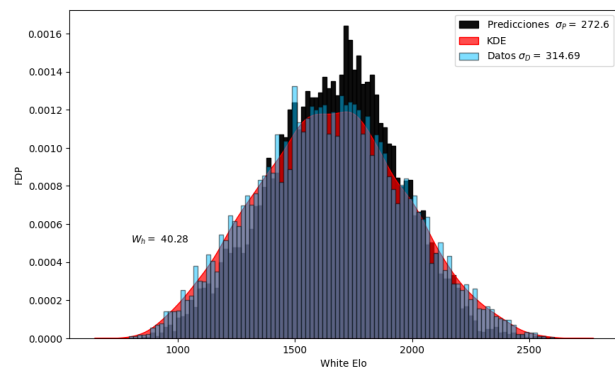


(b)

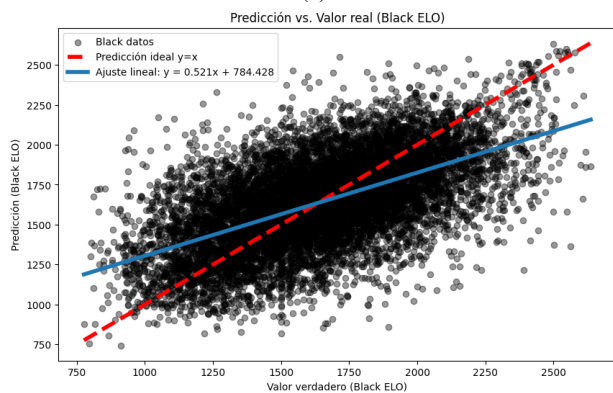
Fig. 10: Kmeans sobre el conjunto de datos.



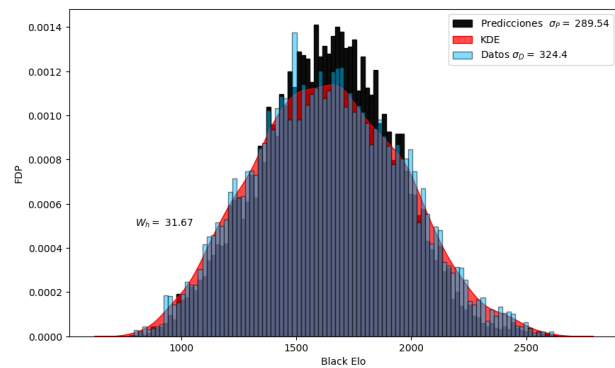
(a)



(b)



(c)



(d)

Fig. 11: Resultados obtenidos en prueba final modelo Deberta.